Test Project Outline – Module A – Mini Test Projects

Competition time

Competitors will have **3 hours** to complete this module.

Introduction

This project consists of several **mini speed Test Projects** that assess key competencies in static website design, front-end development, and basic back-end web technologies. Competitors must implement multiple small, self-contained tasks using the provided media files and starter code. Each mini Test Project focuses on a specific aspect of web development, such as animation, visual effects, interactive components, or simple server-side functionality.

Competitors must follow the original instructions precisely, use the provided assets where indicated, and ensure that each mini Test Project is implemented as an independent, testable unit.

General Description of Project and Tasks

You must create and organize a collection of **mini speed Test Projects** for Module A – Static Website Design. These mini projects cover:

- **Design implementation tasks** using HTML and CSS.
- **Front-end development tasks** with HTML, CSS, and JavaScript.
- **Back-end development tasks** with PHP, involving simple web application behaviour and API-related configurations.

Each mini Test Project is delivered as a separate folder with its own media files and, in some cases, starter code. You must:

- Use the provided media files exactly as referenced.
- Keep each mini Test Project self-contained.
- Include only the required mini Test Project files and folders, and do not add unrelated content.

An index page must link to each mini speed Test Project, allowing easy navigation and overview.

Deliver your work using the following structure, where each folder is named after the identifier of the mini Test Project:

```
/
├── index.html   Index page linking to every mini Test Project
├── A1/          Floating Label
├── A2/          Progress Animation
├── A3/          3D Can Rotation Effect
├── B1/          World's Tallest Buildings
├── B2/          Turntable
├── B3/          Lines and Dots Animation
```

```
├── C1/          Registration Form Validation
└── C2/          API Request Logger
```

The media files of each mini Test Project are provided under `assets/<identifier>/`, for example `assets/A1/`. Copy the media files you need into your own mini Test Project folders.

The back-end mini Test Projects are executed with **PHP 8.x** using the built-in web server, started from the folder of the given mini Test Project:

```
php -S localhost:8080
```

Level System

Each mini Test Project is categorized into one of three difficulty levels, which determine both the expected completion time and point value:

- **Level 1** (L1): Worth 0.75 points and expected to take around 10 minutes of work
- **Level 2** (L2): Worth 1.5 points and expected to take around 20 minutes of work
- **Level 3** (L3): Worth 2.25 points and expected to take around 30 minutes of work

Each category contains minimum 2, maximum 4 mini test projects using one of the following pattern:

- 4 x L1
- 2 x L1 + 1 x L2
- 2 x L2
- 1 x L1 + 1 x L3

This Test Project uses the following composition:

Category	Mini Test Projects	Pattern	Points
Design Implementation	A1 (L1), A2 (L1), A3 (L2)	2 x L1 + 1 x L2	3
Front-end Development	B1 (L1), B2 (L3), B3 (L1)	2 x L1 + 1 x L3	3.75
Back-end Development	C1 (L1), C2 (L2)	1 x L1 + 1 x L2	2.25

Requirements

The following sections describe the individual mini speed Test Projects. Follow all instructions carefully and use the provided media assets as references.

General Requirements

- Create an **index page** that links to each mini speed Test Project.
- The index page must contain a **thumbnail** and **title** for each mini speed Test Project.
- Write at least **one git commit for each mini speed Test Project**.
- Git commit messages must be **readable** and **easy to understand**.

Design Implementation

Tasks in this category recreate a small visual UI or motion effect using **HTML and CSS only**. Match the provided reference media as closely as possible: layout, colour, typography, animation, and CSS-driven interaction such as hover, focus, or checkbox toggles. **JavaScript must not be used** in this category.

A1: Floating Label (Level 1)

Implement a text input with a **floating label** using only HTML and CSS, following the provided reference video.

- Reference video: `assets/A1/input.mp4`

While the input is empty and not focused, the label is displayed inside the input field. When the user focuses the input, the label moves to the top edge of the field, becomes smaller, and changes colour. Once the input contains text, the label stays in the floating position even after the input loses focus.

Requirement notes

- The input field and its label are displayed as shown in `assets/A1/input.mp4`.
- While the input is empty and not focused, the label is displayed inside the input field.
- When the input receives focus, the label animates to the top edge of the field and becomes smaller.
- The label remains in the floating position while the input contains text, even after it loses focus.
- The effect is implemented with HTML and CSS only, without JavaScript.

A2: Progress Animation (Level 1)

Complete the provided page so that a **progress bar** at the top of the viewport reflects the scroll position. Use only HTML and CSS.

- Provided HTML: `assets/A2/index.html`
- Provided CSS: `assets/A2/style.css`

The progress bar width must be **0%** when the page is scrolled to the top and **100%** when the page is scrolled to the bottom. The bar must update as the user scrolls. You may complete the provided `style.css` and, if needed, add CSS-only markup such as a hidden checkbox; you must **not** add JavaScript.

Requirement notes

- The provided `index.html` structure is used as the starting point.
- A progress bar is displayed in a fixed position at the top of the viewport.
- When the page is at the top, the progress bar width is 0%.
- When the page is scrolled to the bottom, the progress bar width is 100%.
- The progress bar width follows the scroll position between those two extremes.
- The effect is implemented with HTML and CSS only, without JavaScript.

A3: 3D Can Rotation Effect (Level 2)

Using CSS only, create the 3D Can Rotation Effect, as shown in the video example.

Initially, the can appears on the center of the screen. The background is `background: radial-gradient(at 50%, #081718 -20%, #28aeb4 90%);`. When the user hovers over the can, it triggers the rotation and scaling animation.

In the media files, you are provided with a video example, a can mockup, and a wrapper that you have to put on the can. Please match the effect seen in the video example as closely as possible.

Requirement notes

- The can is displayed with the wrapper on top.
- The correct colors are set.
- The wrapper is correctly applied to the can.
- The can rotates when you hover over it.

Front-end Development

Tasks in this category add **client-side behaviour with JavaScript**, together with HTML and CSS. The listed interactions must work in the browser without any server-side logic. Use the provided starter code and media files where they are given.

B1: World's Tallest Buildings (Level 1)

You are provided with a JavaScript file containing data about some of the world's tallest buildings, and an image for each tower.

Using that data, dynamically render the buildings on the page as a skyline:

1. Sort the buildings by height, tallest first.
2. Scale each image so the tallest building is 800px tall, and the others are sized in proportion to their real height.
3. Keep each image's aspect ratio.
4. Align all buildings to the bottom of the page.

The result should look like a line of towers standing side by side, with correct scaling and height.

Requirement notes

- The buildings are rendered on the screen.
- The buildings are aligned correctly.
- The buildings are the correct height.

B2: Turntable (Level 3)

Create a fun animation of a **record player** so that the user can listen to the popular song of the week. Use the provided starter page, images, and audio tracks, and follow the reference video.

- Provided HTML: `assets/B2/index.html`
- Provided CSS: `assets/B2/css/style.css`
- Images: `assets/B2/images/` (`armTone.svg`, `circle.svg`, `turntable.PNG`)
- Audio tracks: `assets/B2/audio/DrunkonSoju.wav`, `assets/B2/audio/LostSomewhere.wav`
- Reference video: `assets/B2/video/turntable.mp4`

The provided page already contains a **platter** (`.wheel`), a **tone-arm**, an **Off** button, a **next (>>)** button, and a **volume** range input (minimum 0, maximum 1). Add the JavaScript (the starter HTML links `js/audio.js`; that file is not provided). You may complete CSS classes such as `.animation` that are already defined in `style.css`.

Requirement notes

- When the user presses the platter, the song starts playing, as shown in `assets/B2/video/turntable.mp4`.
- While the song is playing, the platter rotates and the tone-arm moves above the platter, as shown in the reference video.
- The **Off** button stops the song completely, and the platter and the tone-arm return to their original state.
- The volume controller has a minimum value of 0 and a maximum value of 1, and the user can change the volume of the song with it.
- Pressing the **next (>>)** button switches to the other provided audio track.
- When a song ends, the tone-arm animation returns to its original state.
- The turntable works in the browser without server-side logic.

B3: Lines and Dots Animation (Level 1)

You are provided with some basic styles for the website. Using this as a base, try to replicate the lines and dots animation effect you see in the video example.

There are 40 dots. They are randomly positioned and sized when the page is refreshed. The colors are also randomly picked from the provided array.

The effect happens when the user moves the cursor around on the screen.

Requirement notes

- The dots appear randomly in terms of both position and color. The color is chosen from the array.
- The lines and dots animation effect works properly.

Back-end Development

Tasks in this category focus on **server-side or data-driven behaviour**. The visible user interface may be minimal; the marks come from correct output, correct HTTP behaviour, and correct configuration. Each mini Test Project must run with the PHP built-in web server, and you must state in the code or in a `README.txt` which URL the assessor has to call.

C1: Registration Form Validation (Level 1)

Create a form that sends data to the **server** for validation. Use the provided starter files.

- Provided HTML: `assets/C1/index.html`
- Provided CSS: `assets/C1/style.css`
- Provided stylesheet: `assets/C1/bootstrap.min.css`
- Reference image: `assets/C1/capture.png`

Initially, no feedback is shown and no values are entered. If any value is invalid, show feedback and stay on the form. If all values are valid, display `Success`.

Validation rules:

- **First name**: letters `a–z` and `A–Z` only
- **Last name**: letters `a–z` and `A–Z` only
- **Agree to terms and conditions**: must be checked

Validation messages:

- Valid name field: `Looks good!`
- Invalid name field: `Please provide a valid name.`
- Unchecked checkbox: `You must agree before submitting.`

Requirement notes

- The form starts with empty fields and without validation feedback.
- The validation runs on the server side, so invalid data sent directly to the endpoint is also rejected.
- Each valid name field displays the message `Looks good!`.
- Each invalid name field displays the message `Please provide a valid name.`.
- If the checkbox is not checked, the message `You must agree before submitting.` is displayed.
- If every field is valid, the page displays `Success`.

C2: API Request Logger (Level 2)

Create a **PHP endpoint** that accepts a POST request with an `application/json` payload and logs every call into a separate file.

Each time the endpoint is called, the request body must be stored in a text file in the log folder, named `HH-MM-SS-request.txt`, where `HH`, `MM`, and `SS` are replaced with the current hour, minute, and second of the request. Create a `README.txt` file that documents which URL the assessor has to call and where the log files are stored.

No media files are provided for this mini Test Project.

Requirement notes

- The endpoint accepts POST requests with the `Content-Type: application/json` header.
- Each call stores the received request body in a separate text file in the log folder.
- The file name contains the hour, minute, and second of the request in the required format.
- The stored file contains the received JSON payload.
- Requests with another HTTP method or another content type are rejected with an appropriate HTTP status code.
- The `README.txt` file documents the URL to call and the location of the log files.

Other

- This project will be assessed using the **Google Chrome** web browser.

Assessment

The project will be assessed using the provided **marking scheme** and by testing each mini speed Test Project in Google Chrome. Assessors will verify that:

- All required mini Test Projects are implemented and accessible from the index page.
- Functional and visual behaviour matches the descriptions and reference media.
- Technical constraints are respected (e.g. HTML/CSS-only tasks, server-side validation, correct use of the provided data files).
- The user experience is coherent and usable across the mini projects.

Mark distribution

The mark distribution for this project is as follows:

WSOS SECTION	Description	Points
1	Work organization and self-management	2
2	Communication and interpersonal skills	1
3	Design Implementation	3
4	Front-End Development	3.75
5	Back-End Development	2.25
Total		12